API REST de tareas con persistencia, sesiones y tiempo real

Autor: pendiente de completar
Repositorio: https://github.com/wtrujillov1/API
Demostración pública: pendiente de despliegue; interfaz local con `npm start`.

Objetivo y diseño

Se desarrolla una API para que cada usuario registre y gestione sus tareas. La solución integra Node.js 24, Express 5, SQLite y una interfaz web en español. Se elige SQLite por su persistencia en disco sin un servicio externo; el módulo `node:sqlite` permite consultas parametrizadas. La aplicación separa composición HTTP, validación, persistencia y arranque, y mantiene las transformaciones de entrada separadas de las operaciones de E/S.

Contrato REST y validación

Operación	Ruta	Respuesta
Registro / acceso	`POST /api/auth/register` `POST /api/auth/login`	201 / 200
Sesión / cierre	`GET /api/auth/me` `POST /api/auth/logout`	200 / 204
Consulta / creación	`GET /api/tasks`, `GET /api/tasks/:id` `POST /api/tasks`	200 / 201
Actualización	`PATCH /api/tasks/:id`	200
Eliminación	`DELETE /api/tasks/:id?version=N`	204
Tiempo real	`GET /api/events`	Flujo SSE

Una tarea contiene identificador UUID, título de 1–120 caracteres, descripción de hasta 2000, estado (`pending`, `in_progress`, `completed`), versión y fechas; título y descripción rechazan NUL. Las respuestas con contenido usan `{data: ...}`; los errores, `{error: {code, message, details?}}`. Se emplean `400` para entradas inválidas, `401` sin sesión, `404` para recursos no accesibles y `409` para conflictos; también `403` por origen prohibido, `413` por cuerpo superior a 16 KB, `415` por formato o codificación no admitidos y `503` si SQLite está ocupado. Actualizar o eliminar exige la versión conocida y evita sobrescribir cambios más recientes.

Ejemplo: `POST /api/tasks` con `{"title":"Preparar informe"}` devuelve `201` y una tarea de versión `1`. Un `PATCH` con `{"status":"completed","version":1}` devuelve la tarea actualizada. Reutilizar la versión anterior produce `409`. El README incluye peticiones completas con `curl`, cookies y respuestas.

Persistencia, sesión y seguridad

SQLite conserva usuarios, tareas y sesiones entre reinicios. Cada consulta de tareas se restringe al propietario; las consultas SQL utilizan parámetros. Las contraseñas se derivan con `scrypt` y una sal aleatoria. Las sesiones usan una cookie `HttpOnly`, `SameSite=Strict` y `Secure` en producción; al autenticar se renueva el identificador y al salir se invalida. La clave de firma queda fuera de Git; en producción debe proporcionarse de forma segura y tener al menos 32 caracteres.

Comunicación en tiempo real

El cliente abre un `EventSource` autenticado hacia `/api/events`. El servidor emite `task.created`, `task.updated` y `task.deleted` después de las escrituras. Sólo reciben el aviso las conexiones del usuario propietario. La interfaz actualiza el listado al recibir un evento y comunica el estado de conexión. SSE resulta apropiado porque el caso necesita avisos del servidor al navegador; las acciones del cliente permanecen en las rutas REST.

Verificación y reproducción

Desde la raíz: `npm ci`, `npm test` y `npm start`. En este cloud se instala con `npm ci --cache /workspace/.npm-cache`. El servidor escucha por defecto en el puerto 3000 y sirve la interfaz en `/`. Las pruebas de integración usan bases de datos temporales para no alterar los datos de desarrollo. La validación contempla CRUD, errores de entrada, registro y acceso, aislamiento, conflictos de versión, persistencia y recepción de eventos. La instalación reproducible fue verificada y `npm audit` no encontró vulnerabilidades. **Resultado: 13 pruebas aprobadas de 13; 0 fallidas y 0 omitidas.** `npm run check` **también pasó.**

En Chromium se verificaron registro y acceso, CRUD, filtros, cierre de sesión y persistencia. Dos pestañas recibieron eventos de creación, actualización y eliminación. La vista móvil de 390 px funcionó sin desbordamiento ni excepciones JavaScript. Las capturas y el detalle de verificación están en `docs/evidencias/`, como anexos separados. Se validó con Node.js 24.19.0; la demostración web pública queda pendiente de despliegue.

Alcance y entrega

La implementación usa un proceso Node.js y un archivo SQLite. Varias réplicas requerirían distribuir los eventos mediante un servicio compartido. Un despliegue público necesita HTTPS, un proxy de confianza correctamente configurado, almacenamiento persistente y copias de seguridad. El repositorio y las evidencias complementan este informe, limitado a dos páginas. Antes de entregar, completar el autor y renombrar el PDF a **Apellido_Nombre_AplWeb_U2.pdf**; añadir la URL de demostración sólo si está disponible.

Referencias

Brown, E. (2020). *Web development with Node & Express* (2.ª ed.). O’Reilly Media.

Duckett, J. (2014). *JavaScript & jQuery: Interactive front-end web development*. Wiley.

Robbins, J. N. (2018). *Learning web design* (5.ª ed.). O’Reilly Media.